"For the purpose of understanding how both Large Language Models and CAD software interpret STL files."

LLM vs. CAD Software: Comparing Methods

The way an LLM and CAD software handle an STL file comes down to **semantic reasoning** versus **exact geometric reconstruction**.

Feature	LLM Approach	CAD Software Approach
Primary Goal	**Deduction & Intent:** Tries to infer *how* a part was designed (e.g., guessing it's a cylinder with a hole).	**Literal Reconstruction:** Reads coordinates to recreate the exact physical shell out of thousands of triangles.
Data Interpretation	**Text/Tokens:** Reads ASCII as words/coordinates or multi-view images as pixels.	**Matrices/Vectors:** Calculates math instantly via a geometric modeling kernel.
Handling Noise	**Forgiving:** Can look past messy, distorted data or decimals and figure out the ideal shape.	**Strict:** Minor floating-point noise or gaps can break the logic, making the mesh "non-watertight".
Output Type	Parametric, step-by-step code (e.g., CadQuery scripts).	A static, solid mesh or non-parametric B-Rep body.

"STL files exist in two primary formats: ASCII and binary."

the comparison is typically between Binary STL and ASCII STL (sometimes referred to incorrectly as AST).

Both files are exactly the same geometrically—they represent the same 3D object using a mesh of triangles. The key difference lies entirely in how the data is written and stored.

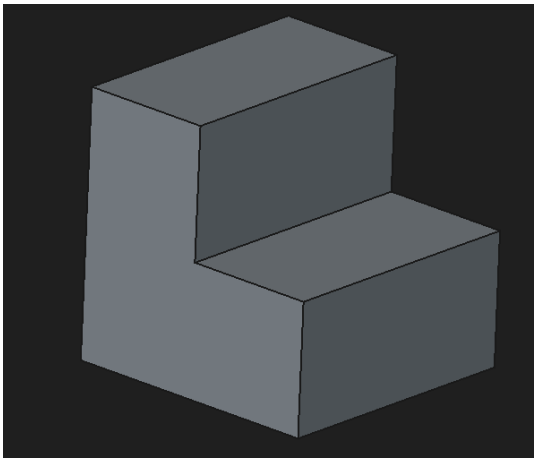
Answer given by ChatGPT:

Yes, an ASCII STL (.ast) file can be read to **infer what the object is**, but only to a limited extent. The file contains only the **triangle mesh** (vertices, facets, and normals) that describes the object's surface, not the original CAD information such as sketches, extrusions, parameters, or feature history. By analyzing the mesh, it is often possible to recognize common mechanical shapes such as cylinders, plates, brackets, holes, and pipes, estimate their dimensions, and infer the likely CAD operations used to create them. However, the exact design intent and construction history cannot be recovered from the STL alone.

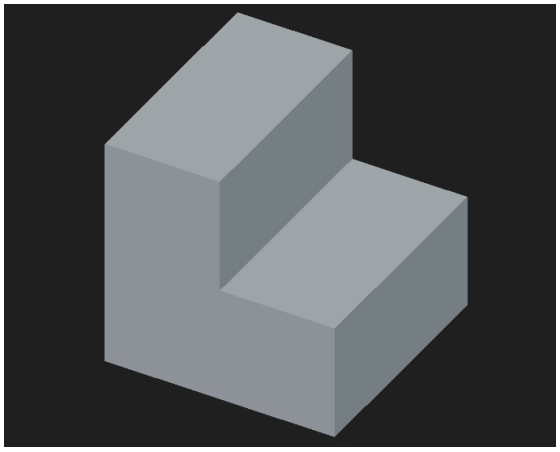
Methods tried:

After creating the 20 heterogeneous CAD parts and their variants, we explored the possibility of reconstructing CAD models using the currently available basic tools. The following approaches were evaluated:

1) Trimesh: [The code for this method was generated by Gemini.]
Trimesh is a Python library for loading and processing triangular meshes, with a focus on watertight surfaces. It can efficiently compute properties such as volume, center of mass, moment of inertia, and surface area. We developed a script that generated geometric information from the input models. This extracted data was then provided to large language models (LLMs) such as ChatGPT and Gemini to generate CadQuery Python scripts, which were subsequently used to create STEP models. This experiment was conducted to evaluate how well the LLMs could understand and reconstruct the original CAD models. The results were promising for simple objects, such as a one-step ladder, but the generated models were not satisfactory for more complex objects.



Step file created by CADQuery

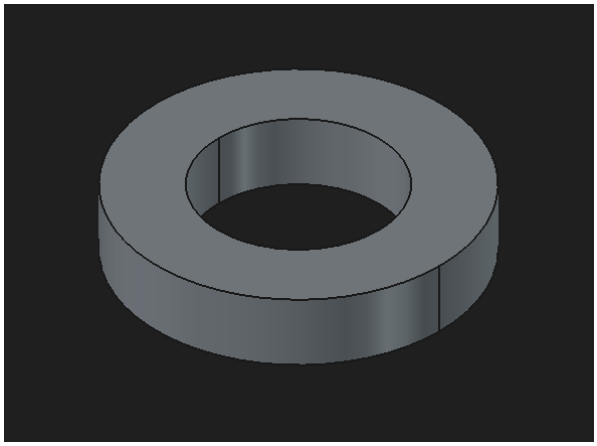


STL file given in script

2) Three.js Viewer JSON Export: [The code for this method was generated by Gemini.]
Three.js represents mesh geometry using an optimized indexed buffer geometry structure, where each unique 3D vertex is stored only once and connected through index arrays to form triangles. This approach is similar to efficient 3D file formats such as OBJ. The organized representation of matrices, vectors, and vertex normals makes the data easier to process using machine learning models, such as Graph Neural Networks (GNNs) and Point Transformers, compared to parsing verbose STL text containing commands like facet normal and outer loop.

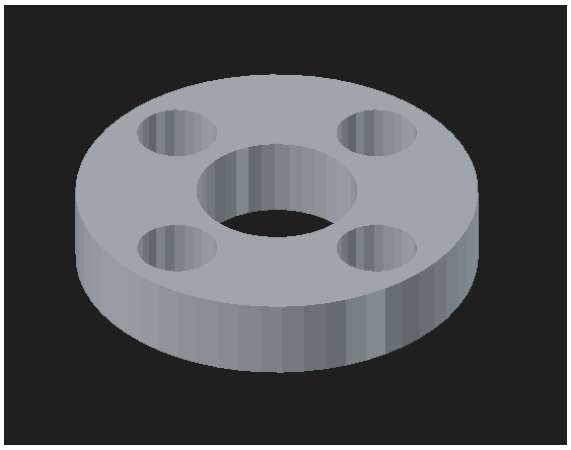
Script Name: json-metadata-extraction

This Python script loads a Three.js JSON mesh file, extracts the vertex coordinates, and groups them into 3D (x, y, z) points. It computes the mesh's bounding box, samples every 50th vertex to create a compact representation, and generates a structured JSON summary containing the mesh type, vertex count, triangle count, bounding box dimensions, and a subset of vertex coordinates. This concise representation enables efficient analysis and serves as suitable input for LLMs.



Step file created by CADQuery

(circular_flange_reconstructed.step)



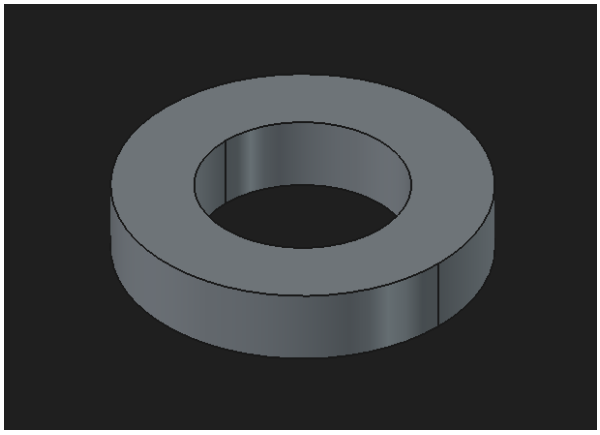
STL file given in 3js viewer

(15-Body.stl)

3) Feature Extraction Script: [The code for this method was generated by Gemini.]
When working with text and JSON data, using a random sample of vertices is not sufficient to accurately represent a 3D model. Instead, a Python script using libraries such as **Trimesh** can be used to mathematically extract meaningful geometric properties and provide the LLM with a clean, structured summary. This approach transforms the JSON output from a collection of random vertex coordinates into a comprehensive geometric description of the model.

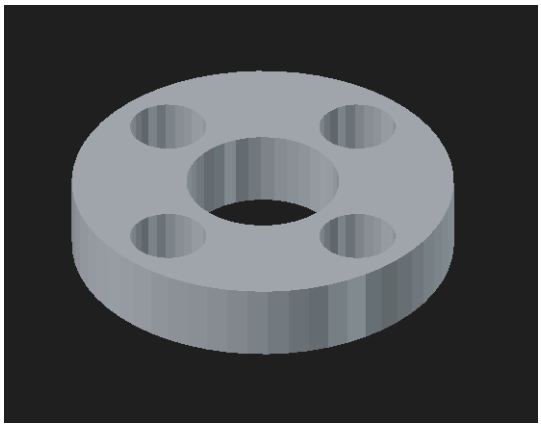
Script Name: Feature Extraction Script

This Python script uses the **Trimesh** library to load an STL file and extract important geometric properties of the 3D model. It computes global metrics such as the bounding box, overall dimensions, number of vertices, number of triangles, surface area, and volume (for watertight meshes). The extracted information is organized into a structured JSON payload, which is both printed and saved to a file. This concise, LLM-friendly representation enables efficient analysis and improves the model's ability to understand the geometry of the input mesh.



Step file created by CADQuery

(deduced_flange.step)



STL file given in script

(15-Body.stl)

4) Open3D:

Script Name: Advanced Extraction Script
The Advanced Extraction Script is a Python-based solution designed to automatically reverse-engineer and extract structured geometric information from 3D STL files, making the data easier for an AI model to interpret. It uses the **Open3D library** and the RANSAC (Random Sample Consensus) algorithm to analyze the geometry. The script converts the STL mesh into a point cloud and iteratively detects geometric primitives such as planes, cylindrical surfaces, and holes. Instead of producing thousands of raw vertex coordinates, it generates a clean JSON file containing the detected geometric features, their parameters, and dimensions. This structured representation enables LLMs to better infer the original design intent and generate accurate parametric CAD code, such as CadQuery scripts, for recreating the model.

The LLM did not generate any code. Instead, ChatGPT provided the following explanation:

The current Advanced Extraction Script focuses on recognizing 3D geometric primitives (such as planes, cylinders, and cones), whereas CadQuery is based on a sketch-driven modeling approach rather than primitive-based geometry.